

Titanic PyTorch 任务学习笔记

基于最新 *tainiTrain.py* 与 *predict.py* 整理

一、任务简介

这次任务的目标是使用 PyTorch 做一个 Titanic 生存预测模型。输入一名乘客的基本信息，模型最后输出这名乘客是否存活的预测结果。这里的 Survived 是标签，0 表示没有存活，1 表示存活。

这次任务我自己使用 torch.nn.Module 定义神经网络，自己完成数据清洗、特征处理、训练、验证、测试、模型保存以及新乘客预测。

二、整体项目结构

目前项目主要分成 src 和 docs 两个文件夹。src 里面放程序和数据，docs 里面放学习笔记以及训练过程中生成的图片。

```
LR/
├── src/
│   ├── tainiTrain.py
│   ├── predict.py
│   ├── train.csv
│   ├── titanic_model_best.pth
│   └── requirements.txt
└── docs/
    ├── loss_curve.png
    ├── accuracy_curve.png
    └── 学习笔记
```

三、tainiTrain.py : 训练模型

1. 导入需要的库

我首先导入 Pandas、PyTorch、神经网络模块、DataLoader，以及 matplotlib。

Pandas 主要负责处理表格数据，PyTorch 负责建立和训练神经网络，matplotlib 用来画训练曲线。

2. 固定随机种子

我设置 `SEED = 42`，并使用 `torch.manual_seed(SEED)`。这样程序再次运行时，随机划分数据和模型初始化等过程更容易得到相同的结果，方便复现实验。

3. 使用相对项目位置读取文件

我通过 `Path(_file_).resolve().parent` 等方法找到当前项目的位置，再确定 `train.csv`、模型文件和 `docs` 文件夹的位置。比直接写死电脑上的某一个盘更加方便，因为项目换到其他电脑后，只要文件夹结构不变，一般就可以继续运行。

4. 读取 Titanic 数据

使用 `pandas.read_csv()` 读取 `train.csv`。读取以后，`df` 就是一个表格数据。

5. 特征工程

原始数据有一些信息不能直接很好地交给神经网络，所以我根据已有字段构造了一些新的特征。

Deck：从 `Cabin` 中取出船舱甲板的第一个字符。缺少 `Cabin` 时使用 `Unknown`。因为我认为后面的数字对模型的学习并没有多大帮助。

FamilySize：家庭人数，计算方法是 `SibSp + Parch + 1`。因为我认为这个数据更直接简洁。

IsAlone：如果 `FamilySize` 等于 1，就记为 1，否则记为 0。

Title：从 `Name` 中提取称呼，例如 `Mr`、`Miss`、`Mrs` 等。因为我认为名字本身对生存概率影响不大，而不同称谓可能和乘客的年龄、性别以及社会身份有一定关系，因此把它单独提取出来作为一个分类特征。

FarePerPerson：船票价格除以家庭人数，用来表示平均每个人承担的票价。这样可以避免模型只看总票价，便于模型学习理解。

这些特征的作用是尽可能从原始数据中提取出一些对预测可能有帮助的信息。

6. 划分训练集、验证集和测试集

这是我这次修改中比较重要的一部分。以前的代码直接使用测试集来挑选最佳模型和进行早停，这样会让测试集参与模型选择。修改以后，数据按照 64% 训练集、16% 验证集、20% 测试集进行划分。

训练集：真正用来更新神经网络参数。

验证集：训练过程中用来观察模型效果、选择最佳模型以及决定什么时候早停。

测试集：直到训练结束、最佳模型已经确定以后，才进行最终评价。

这样可以避免测试集提前参与模型选择，使测试集真正起到最后检查模型泛化能力的作用。

7. 缺失值处理

Age 存在缺失值，所以程序使用训练集 Age 的中位数进行填充。Embarked 的缺失值则使用训练集中的众数进行填充。这里特意只从训练集计算这些参数，再把相同参数用于验证集、测试集和新乘客。

这样做可以避免测试集或验证集的信息提前进入训练过程，也就是减少数据泄漏。

8. 类别数据的处理

Sex、Embarked、Deck 和 Title 属于类别型数据，神经网络不能直接把这些文字当成普通数字处理。程序先根据训练集确定每个类别有哪些取值，再进行 one-hot 编码。

验证集、测试集和新乘客也必须使用训练集确定好的类别和列顺序。

9. 数值特征标准化

Pclass、Age、SibSp、Parch、Fare、FamilySize、IsAlone 和 FarePerPerson 等数值特征会进行标准化。程序只使用训练集计算均值和标准差，然后用这些值处理训练集、验证集、测试集以及之后的新乘客。

这样做可以让不同数值范围的特征处在比较接近的尺度上，有利于神经网络训练。

10. Dataset 和 DataLoader

处理好的数据会转换成 PyTorch Tensor，然后使用 TensorDataset 和 DataLoader。Dataset 可以理解成数据集的封装，DataLoader 可以按照 batch 一批一批地把数据送进神经网络。

这符合 minibatch 训练方式，而不是一次把所有训练数据同时用于一次参数更新。

11. 自定义 TitanicClassifier

模型继承 torch.nn.Module。网络结构为：输入层 → 16 个神经元 → 8 个神经元 → 1 个输出。中间使用 ReLU 激活函数。最后输出一个数值，因为这是二分类问题。

最后一层没有直接使用 Sigmoid，是因为训练时使用 BCEWithLogitsLoss，它内部已经包含了适合二分类的处理。进行预测或者计算准确率时，再通过 torch.sigmoid() 把输出转换成 0 到 1 之间的概率。

我的小心思：这里没有直接把输入特征数量写成一个固定数字，而是让程序根据训练数据自动获取。因为前面进行了独热编码，如果分类变量的类别发生变化，最终特征数量也可能发生变化，所以自动读取输入维度会更加方便。

12. 损失函数和优化器

损失函数使用 BCEWithLogitsLoss，适合二分类任务。优化器使用 Adam，学习率设置为 0.001。训练时根据损失计算梯度，然后由 Adam 更新网络参数。

13. model.train() 和 model.eval() 的区别（我的重新思考）

训练阶段使用 model.train()，表示进入训练状态。验证和测试时使用 model.eval()，表示进入评估状态。

这次准确率的计算专门写成 evaluate_accuracy() 函数，并在函数内部先调用 model.eval()，然后再计算准确率。这样可以避免像 Dropout 这类训练时和测试时行为不同的层影响模型比较。

14. 如何选择最佳模型和早停

每训练完一个 epoch，就在验证集上计算一次准确率。如果验证集准确率比之前的最好结果更高，就保存当前模型参数。

如果验证集准确率连续很多轮没有提升，就触发 early stopping（早停）。当前 patience 设置为 20，也就是验证效果 20 轮没有改善时就提前结束训练。可以节省很多算力资源，避免模型一直进行没有明显效果的训练，同时也可以节省时间。

最关键的一点是：现在最佳模型和早停依据都是验证集（知错就改），而不是测试集。

15. 最后的测试集评估

训练结束后重新加载保存下来的最佳模型，再使用 test_loader 计算一次测试准确率。

16. 保存模型

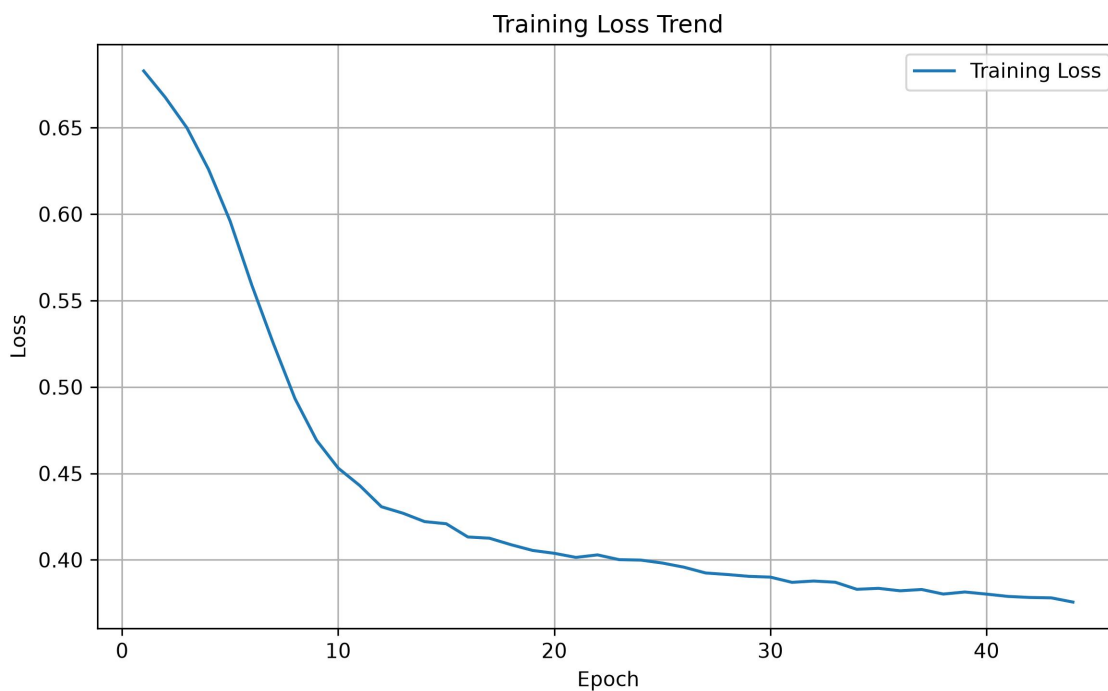
程序保存的不只是 model_state_dict，还保存了 input_size 和 preprocessing 信息。preprocessing 中包括训练集得到的缺失值处理参数、类别映射、特征列顺序、训练集均值和标准差等。

保存这些信息非常重要，因为以后 predict.py 输入一个新乘客时，必须按照训练模型时完全相同的方式处理数据。

17. 训练曲线

我把训练过程中的 loss 保存成 loss_curve.png，并把训练集和验证集准确率保存成 accuracy_curve.png。通过曲线可以观察模型训练过程中是否逐渐收敛，以及验证集表现有没有继续提升。

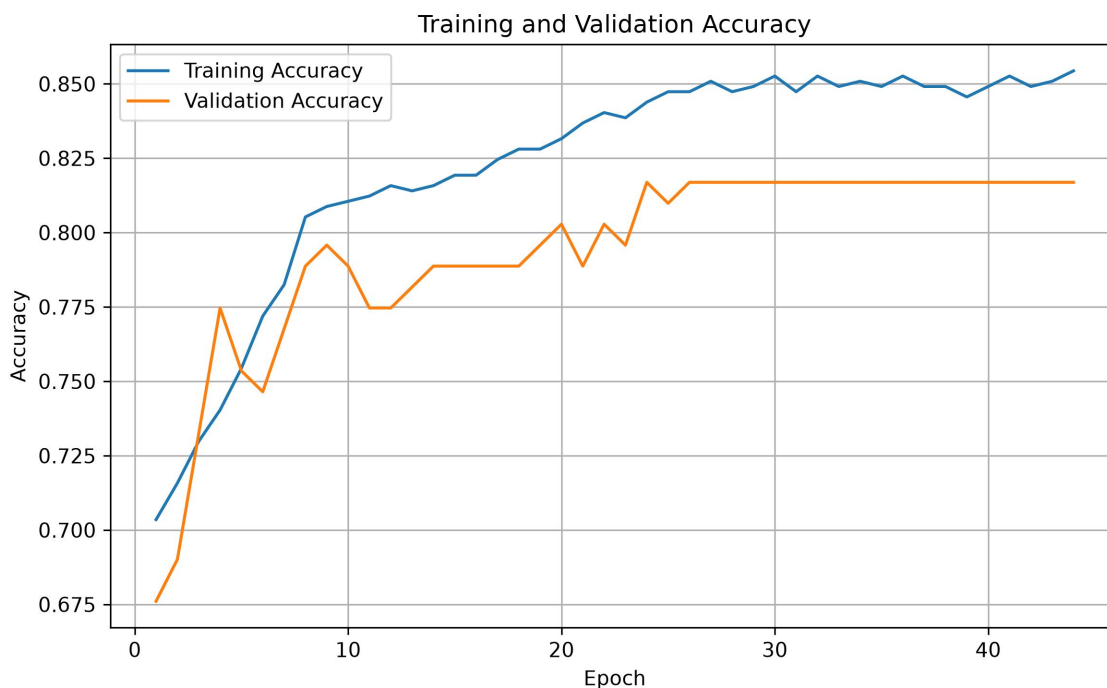
当前训练得到的 Loss 曲线：



从这张图可以看出，训练 Loss 在前面的训练阶段下降比较明显，后面的变化速度明显变慢。虽然中间存在一些小幅度的上下波动，但是整体趋势仍然是在下降。

这说明模型在训练过程中确实在逐渐调整参数，使自己的预测结果和训练数据中的真实标签越来越接近。后期 Loss 的下降幅度变小，也说明模型已经学到了一部分比较稳定的规律。

当前训练得到的准确率曲线：



这张图同时显示训练集准确率和测试集准确率。可以看到，训练初期训练准确率从比较低水平快速上升，之后逐渐变得平稳。测试准确率同样在前期快速提高，之后主要在较高水平附近上下波动。

从后半段可以看到，测试准确率已经没有持续上升，而是出现一定波动。这也是我设置 Early Stopping 并保存最佳模型的一个原因：不能简单地认为训练轮数越多，模型在测试数据上的表现就一定越好。

训练中的小插曲

一开始我用的其实是 1000 轮，但我发现准确率在 100 轮之后就一直在较高水平上下波动，没有明显的提高。甚至还出现了训练集准确率高于测试集的现象，我觉得此时模型已经出现了过拟合的危机，所以我果断减少了训练轮次。

四、predict.py：使用训练好的模型预测新乘客

1. 单独写 predict.py 的原因

训练模型和使用模型是两个不同的过程。trainTrain.py 负责训练并保存模型，predict.py 负责加载已经训练好的模型并进行新乘客预测。这样项目结构更加清楚，也方便以后单独使用预测功能。

2. 加载模型

predict.py 会读取 titanic_model_best.pth，重新创建和训练时一样的 TitanicClassifier，然后使用 load_state_dict() 加载保存下来的模型参数。最后调用 model.eval() 进入预测状态。

3. 预测时必须使用训练阶段保存的 preprocessing

新乘客不能重新计算自己的标准化参数，否则输入数据和模型训练时的数据尺度就不一致。

所以 predict.py 直接读取模型文件中保存的 preprocessing 信息，并按照训练时的规则处理新乘客。

4. predict_passenger() 的任务

函数接收 Pclass、Sex、Age、SibSp、Parch、Fare、Embarked、Deck 和 Title 等信息。首先重新计算 FamilySize、IsAlone 和 FarePerPerson，然后进行缺失值处理、类别编码、特征列对齐和数值标准化。处理完成后转换成 Tensor，最后送进神经网络。

5. Sigmoid 输出存活概率

模型最后输出的是一个 logit。通过 torch.sigmoid() 后，结果会变成 0 到 1 之间的数。由于训练标签中 1 代表存活，所以这个数可以理解成模型预测该乘客存活的概率。

6. 交互式输入

运行 predict.py 后，可以在终端输入乘客信息。程序对 Pclass、Sex、Age、SibSp、Parch、Fare、Embarked、Deck 等输入进行检查。输入不符合要求时给出提示，而不是直接让程序因为 ValueError 等异常退出。

五、这次我根据意见进行修改

- 一、增加验证集，不再直接使用测试集挑选最佳模型。
- 二、最佳模型根据验证集准确率保存。早停根据验证集表现决定。
- 三、测试集只在训练结束后进行最终评价。
- 四、训练、验证、测试准确率的计算统一通过 `evaluate_accuracy()`，并在 `eval()` 状态下完成。
- 五、保存模型时继续保存训练阶段得到的 `preprocessing` 信息，保证 `predict.py` 与训练过程使用完全一致的预处理。

六、我对整个任务的理解

我现在对这个任务的理解是：首先把 Titanic 原始数据整理干净，然后从原始字段中构造一些新的特征，再把文字类别转换成 one-hot，把数值进行标准化。接下来把数据分成训练集、验证集和测试集。

训练集负责让模型学习，验证集负责在训练过程中选择比较合适的模型和决定是否早停，测试集最后才使用，用来检查模型最终的预测效果。

训练完成以后，把模型参数和预处理所需要的信息一起保存。预测新乘客时，先按照训练时相同的方式处理输入，再交给神经网络进行预测。

刚开始看这个任务时，我觉得重点只是定义一个神经网络，但真正做下来后才发现，数据预处理同样占了很大一部分工作，以及交互式预测的部分。另外，我也明白了训练完成以后还要保存预处理参数的重要性。

总体来说，这次实验让我对一个比较完整的 PyTorch 二分类项目有了更具体的认识，也把之前学到的数据集、特征工程、DataLoader、损失函数、优化器、反向传播和模型预测等知识联系了起来。